

My Module: Staff Dashboard (End-to-End) — 2-Day Build Plan

Scope: Full stack — backend APIs and frontend UI for the Staff role only.

0. Requirements & Specifications

Functional Requirements

ID	Requirement
FR-1	Staff can log in and see only staff-scoped routes
FR-2	Staff sees a queue of applications assigned to them (status = <code>under_review</code>)
FR-3	Staff can open an application and view student details + all uploaded documents
FR-4	Staff can mark each document individually as Valid / Invalid / Needs Resubmission
FR-5	System auto-checks basic eligibility (marks/income vs scheme criteria) and shows a pass/fail flag staff can see before deciding
FR-6	Staff can add remarks per document and an overall remark for the application
FR-7	Staff can submit a final recommendation: "Recommend Approval" or "Recommend Rejection"
FR-8	On recommendation, application status updates and (per Section 3 of the shared workflow doc) either routes to Admin or auto-approves, depending on the scheme's <code>requiresAdminApproval</code> flag
FR-9	If a document is marked invalid, student is notified and can resubmit — application returns to staff queue automatically
FR-10	Staff can see their own basic stats (applications processed, average turnaround)

Non-Functional Requirements

ID	Requirement
NFR-1	All staff API routes protected by role middleware (<code>role === 'staff'</code>), server-side
NFR-2	Every verify/recommend action writes an entry to <code>AuditLogs</code>
NFR-3	Queue is paginated and filterable (by scheme, by date)
NFR-4	Document viewer must handle common formats (PDF/image) without requiring download

What You're Explicitly NOT Building

- Scheme creation/editing (Admin module)
- Final admin approval/override/disbursement (Admin module)
- Student application form itself (Student module) — you only ever *read* what they submitted
- Notification delivery mechanism — you just trigger `createNotification()`

1. Module Breakdown

- **Module A** — Review Queue (list + filters)
- **Module B** — Document Verification (per-document valid/invalid + remarks)
- **Module C** — Eligibility Auto-Check Display
- **Module D** — Recommendation Submission (approve/reject → routes to Admin or auto-approves)
- **Module E** — Staff Stats (lightweight, own performance only)

2. Day-by-Day Task List

DAY 1 — Morning (Setup + Module A)

- ☐ Sync with Auth owner: confirm JWT payload and reusable role middleware
- ☐ Confirm `Application` schema fields with the team — especially `documents[]` structure and `assignedStaffId`
- ☐ Confirm how assignment works: auto-assign on submission, or staff "claims" from a shared pool? Decide with team now — it changes your query
- ☐ Build API: `GET /api/staff/applications?status=under_review` (paginated, filterable)
- ☐ Frontend: Queue table (student name, scheme, submitted date, status badge)

DAY 1 — Afternoon (Module B)

- ☐ Build API: `GET /api/applications/:id` (full detail incl. documents)

☐ Build API: `(PATCH /api/applications/:id/documents/:docId)` (set status: valid/invalid/needs_resubmission + remark)

☐ Frontend: Application detail view — student info panel + document list with inline preview (PDF/image) and per-doc status buttons

☐ Frontend: Remark input per document

DAY 1 — End of day checkpoint

☐ Demo queue + document review to team lead with dummy data — confirm document upload URLs from Student module actually render correctly here (common integration snag)

DAY 2 — Morning (Module C + D)

☐ Build eligibility check logic: compare `(application.formData)` against `(scholarship.eligibilityCriteria)` (e.g., $\text{marks} \geq \text{minMarks}$, $\text{income} \leq \text{maxIncome}$) — returns pass/fail + which rule failed

☐ Frontend: Show auto-check result as a banner ("  Meets criteria" / "  Income exceeds limit by ₹X")

☐ Build API: `(PATCH /api/applications/:id/recommend)` (body: `{ decision: 'approve' | 'reject', remark }`)

☐ Logic: if `(scholarship.requiresAdminApproval === false)` and decision is approve → status jumps straight to `(approved)`; else → `(pending_admin_approval)`

☐ Frontend: Recommend button with confirmation modal

DAY 2 — Midday (Module E + resubmission loop)

☐ Build API: `(GET /api/staff/stats)` (count processed, avg turnaround)

☐ Frontend: Small stats card on dashboard header

☐ Verify resubmission loop: when student re-uploads after "invalid" flag, application returns to `(under_review)` and appears back in your queue — test this path explicitly, it's easy to miss

DAY 2 — Afternoon (Integration + Polish)

☐ Full integration test: student submits → appears in your queue → you verify → recommend → confirm it lands correctly in Admin's queue (or auto-approves)

☐ UI polish: status badges with color coding, loading/empty states

☐ Responsive check

DAY 2 — Final hours

☐ Deploy/confirm merge

- ☐ Prepare demo segment: show a document being flagged invalid → student notified → resubmitted → re-verified → recommended. This "loop" is the most convincing part of your module to show live.
- ☐ Rehearse: *"How do you decide who processes which application?"* and *"What happens after you recommend?"*

3. API Contract You Need From Teammates (Day 1 AM handshake)

From...	What
Auth owner	JWT structure, role middleware
Student module owner	Exact <code>documents[]</code> field names/URLs so your preview component works without guessing
Admin module owner	Confirm <code>requiresAdminApproval</code> flag exists on <code>Scholarship</code> and confirm the exact status string used for "goes to admin" so your PATCH writes the right value

4. Your API Endpoints — Full List

```
GET    /api/staff/applications?status=under_review
GET    /api/applications/:id
PATCH /api/applications/:id/documents/:docId
PATCH /api/applications/:id/recommend
GET    /api/staff/stats
```